Computer Games Development
Project Report
Year IV

Jakub Stepien
C00284361
13/04/2026

Contents

Acknowledgements 2
Project Abstract 3
Project Introduction and/or Research Question 3
Why choose a 2D platformer game? 3
What are the objectives of this project? 3
Research Questions 3
Main Contributions of the project: 3
Literature Review 4
Evaluation and Discussion 4
Conclusions 6
References 6

Acknowledgements

I would like to thank my project supervisor Omer Ali for the positive feedback and helpful ideas on how to improve the game throughout the course of this project. I would also liketo thank the other supervisors, Ben O'Shaughnessy, Libor Zachoval, Amr Abdelhafez and Noel O'Hara for all the constructive criticism that I took and improved upon, and to Martin Harrigan for organising and manage this entire project.

Project Abstract

Backtrack is a 2D platformer game where the game begins with the main character just completing a difficult quest and is on his way home through a thick forest. The player will have to navigate the tough terrain and use their abilities to their own advantage.

The game is built using C++ and SFML 3.0 with smooth movement, an inventory system, an level editor/creator and a player state machine to manage logic and animations.

Project Introduction and/or Research Question

Why make a game using only C++ and the SFML library?

I chose to make the game using SFML because it's what I've programmed the most in. Since first year, I've used it to make different games, and it is what I know the most about at this point in time. And I think it's interesting to see what I can make with just this library.

Why choose a 2D platformer game?

Platforming games are some of my favourite games, both 3D and 2D. I've always been fascinated by games with incredibly smooth movement that are very satisfying to play and feel rewarding to just playing. This made me want to make a game where the movement would be the key mechanic and also with an inventory system, the player would pick up items that would alter the movement mechanics even more.

What are the objectives of this project?

My objectives for this project are to make a good movement system, create an inventory system, and explore how level editors are made well and how auto-tiling works.

Research Questions

What makes good movement in a game so satisfying?

How does auto-tiling help in the creation of levels?

Main Contributions of the project:

While working on this project, I made a player with a state machine to handle states and animations, and a good core movement system. I realised that a bit part of making a game feel smooth and satisfying is a good balance of how it plays and how it looks visually. Making a system to handle animations work in tandem with my player states, as I used the change in states to transition easily from one set of frames to another.

I made a working inventory system where the player could check their items, and info about the items. Where initially I thought this would be tricky to do using just the SFML library, it ended up not being nearly as difficult as I imagined. The inventory system helps explain certain aspects of the game to the player or remind them of those elements. For example, having an item that grants the player double jump, after a few levels of not using it, someone might forget they have it without any information, whereas with an item they can see it in the inventory, hover over it, and see what it does.

I also explored level editors and made my own version of an auto-tiler. This made making levels so much easier and cleaner since I didn't have to worry about many different types of tiles and managing the corners, or edges. After I set up the auto-tiler, I could simply place a grass tile next to another grass tile, and they would match perfectly side by side. It also made the level-making so much faster since I made levels that were 2D arrays save to a text file, without an editor I would have to manually changing the numbers in the text files and that would be difficult to visualise from just numbers.

Literature Review

The key concepts of a platforming game include precise movement, a good collision system and level design that is challenging. I think great examples of these concepts can be found in games like Celeste which is known for its very precise and fluid movement, or like Hollow Knight where any wrong move can set you back to the last checkpoint. While I tried to focus on the core movement to be in similar satisfaction to that of the movement in Celeste, I also tied in an inventory system that would give the player more control over their character similar to the inventory system in Hollow Knight.

Evaluation and Discussion

From exploring other games with a similar premise, satisfying movement definitely comes from a balance of how good it feels to play along with how good it looks visually. A game could have the smoothest logic and movement but with the wrong animation and visuals, it wouldn't look as satisfying or might even seem clunky at times. The same way a game could have the best animations, with hundreds of frames per animation, but with choppy logic that makes the character stutter, it would feel more unsatisfying than smooth or rewarding.

In the long run the level editor has proved to be extremely useful, and also pretty satisfying to use itself. Seeing how the tiles match together and visually seeing the level in front of me was much better than looking at an array of numbers.

Project Milestones

- Player with core movement mechanics
- Player state machine
- Animation loader/handler
- Level editor
- Auto-tiler
- Tile based levels
- AABB collision system
- Inventory system
- Health system
- Items

Major Technical Achievements

I think my major technical achievements was the inventory system with the items, level editor with the auto-tiler and the player state machine with animations.

Project Review

From the start I was wondering if I should swap the state machine to its own class, but I only ended up using it for the player, so I left it as a part of the player's logic, which I think was a better choice than trying to separate it into its own class since I think that would've cause a lot of issues that would take me some time to fix. The animation loader worked perfectly from the start, the only thing I had to add when I added a new state was to add the frames which was very simple to do with how I set it up.

I was struggling for a while with the collision system because I didn't realise that SFML had changed some of their function names, so I initially tried checking for specific points on the player and seeing if they collided with tiles, but that was a lot of checks that got very confusing very quickly. Later I realised that the only changed the name of the function to find the overlap between two IntRects instead of removing it completely and with that I refined the collision system to use AABB.

If I were to start again, I would definitely start with things that could be modular straight away, so instead of starting with a state machine specifically in the player class, I would make a state machine class and have the player have a state machine object. Making it easier for the long run, as opposed to doing what is easier in the moment.

Conclusions

In conclusion, I think for the player to have a truly satisfying and rewarding experience from just playing the game, the game must deal with high-functioning logic, have good and smooth visuals and the levels and game must also be designed well.

Future Work

Some improvements that I can already think of that could be beneficial for the future include:

- Using a resource holder to manage all the texture and sprites.
- Larger sized levels to make better designed/more challenging levels.
- More movement types to give the player a wider range of movement combinations.
- Making a visual menu in the level editor for selecting tiles.

References

Maddy Makes Games (2018). Celeste [Video Game]

Team Cherry (2017). Hollow Knight [Video Game]